High-priority improvements These are the improvements I would prioritize. 1.1 Improve narrative understanding using semantic retrieval High priority Moderate effort

Current implementation:

The synthesizer retrieves relevant screenplay elements using simple, case-insensitive keyword and character-alias matching.

For example, it might search for:

- Alexander
- Duke of Ashburn
- Signet ring
- Marriage contract The problem is that screenplay events aren't always described using identical words.

Suppose Episode 7 says:

"The soldier examines the royal signet."

Episode 42 says:

"The Duke's ancestral seal is presented before the court."

A keyword search may fail to recognize the relationship between these two events.

Proposed improvement: Semantic retrieval using embeddings.

Generate embeddings for narrative moments and compare them using cosine similarity.

$$\text{sim}(A, B) = \frac{A \cdot B}{\|A\|\|B\|}$$

The synthesizer can retrieve semantically related moments even when their wording differs.

For the lean implementation:

- Generate embeddings only for extracted narrative moments, not all screenplay text.
- Store the embeddings in memory or a local JSON file.
- Retrieve the top 3–5 related moments.
- Pass the retrieved original screenplay elements to the LLM. Improved retrieval pipeline Narrative moments

Embedding generation

Semantic similarity search

Retrieve original screenplay elements

LLM cross-episode synthesis

Why it matters: Better narrative connections without introducing an elaborate knowledge graph. 1.2 Introduce a diversity-aware final ranking High priority Low effort

Current implementation:

The system sorts candidates based on four rubric scores and returns the top 15.

However, consider this situation:

- 10 recommendations are from Episode 42.
- 3 recommendations are from Episode 41.
- 2 recommendations are from Episode 40. All 15 may be individually good, but the final recommendations are heavily concentrated around one storyline.

This reduces the variety of promotional options available to the marketing team.

Proposed improvement: Diversity-aware selection.

After ranking, introduce a simple selection constraint:

- Avoid selecting too many clips from the same episode.
- Preserve different promotional archetypes.
- Encourage different character combinations and storylines.
- Retain the original quality ranking as the primary criterion. For example: Constraint Suggested rule Episode concentration Maximum 3 recommendations per episode Archetype diversity Prefer representation of all 3 archetypes Composition diversity Include both single and multi-episode promos Quality Never select a low-quality clip just to satisfy diversity

These are configurable targets, not universal optimal thresholds.

You could implement this with a greedy selection algorithm instead of introducing a complicated ranking formula.

Why it matters: The marketing team receives a varied collection of promotional ideas rather than 15 variations of essentially the same scene. 1.3 Improve LLM output reliability High priority Low effort

Current implementation:

The system expects the LLM to return structured JSON containing candidate IDs, timestamps, scores, and explanations.

But LLM responses can contain:

- Invalid JSON.
- Missing mandatory fields.
- Incorrect element IDs.
- Unexpected enum values.
- Truncated responses.
- Scores outside the permitted range. Proposed improvement: Structured output validation and controlled retries.

Use a JSON Schema or Pydantic model to validate every response.

For example: class PromoCandidate(BaseModel): start_element_id: str end_element_id: str hook_timestamp_offset_seconds: float archetype: Literal["immediate_shock_opener", "visual_anomaly_mystery_escalation", "emotional_reversal_turning_point"] standalone_coherence: int = Field(ge=1, le=5) hook_strength: int = Field(ge=1, le=5) cliffhanger_strength: int = Field(ge=1, le=5) emotional_impact: int = Field(ge=1, le=5)

If validation fails:

1. Capture the original response.
2. Identify the schema errors.
3. Retry once with a concise correction prompt.
4. Reject and log the response if it still fails. Use provider-supported structured output where available.

Why it matters: Prevents one malformed LLM response from breaking the entire pipeline. 1.4 Improve timestamp accuracy and editing instructions High priority Low–moderate effort

Current implementation:

The parser uses element-level timestamps and integer centiseconds.

This is reliable for selecting complete screenplay elements, but it has a limitation.

Suppose a dialogue element lasts 8 seconds: [00:24.10 - 00:32.10]

GERALD: "I cannot wed a woman whose family name is ruined. You are no longer welcome here."

The most powerful promotional line may be the first sentence.

Using the complete element could include unnecessary dialogue.

Proposed improvement: Add precise trim recommendations.

Your plan already includes trim_quote_hint for internal sentence-level cuts. You can make this more useful by having the LLM return:

- Exact opening dialogue.
- Exact ending dialogue.
- A verbatim trim quote.
- Whether the trim is at an element boundary or internal to an element.
- Whether the cut requires manual editor verification. Do not calculate timestamps by interpolating character positions. Without actual audio/video timing, character-based interpolation is unreliable.

Why it matters: The recommendations become more actionable for the video editing team. 1.5 Improve evaluation with a small human-labelled benchmark High priority Moderate effort

Current implementation:

The document proposes manually auditing 5–10 episodes and the top 10–15 recommendations.

This is useful, but it doesn't establish how accurately the system discovers the best promotional moments.

Proposed improvement: Create a small reference dataset.

Ask a human reviewer to manually identify approximately 20–30 strong promotional moments from representative episodes.

Then compare the system's recommendations against those references.

Measure: Metric What it tells you Precision@15 How many top 15 recommendations are genuinely good? Recall@15 How many known strong moments were discovered? Timestamp accuracy How closely do recommendations match reference boundaries? Duplicate rate How many recommendations are repetitive? Human acceptance rate How many recommendations would an editor actually use?

For example:

$$Precision@15 = \frac{\text{Relevant recommendations in Top 15}}{15}$$

This is more meaningful than simply reporting that the pipeline successfully processed all 50 episodes.

Why it matters: You can demonstrate whether the system actually solves the promotional discovery problem, rather than merely producing valid JSON.